PostgreSQL Best Practices & Compliance Standards

Document Information

Field	Value
Document Name	PostgreSQL Best Practices & Compliance Standards
Version	1.0
Prepared By	Database Engineering Team
Scope	Production PostgreSQL Environments
Database Version	PostgreSQL 13+
Review Frequency	Quarterly

---

1. Introduction

This document defines the PostgreSQL database standards and operational best practices that must be followed across all environments including Development, QA, UAT, and Production.

The purpose of this document is to:

- Standardize PostgreSQL deployments
- Improve database performance and reliability
- Ensure security compliance
- Reduce operational risks
- Enable proactive monitoring and maintenance
- Support high availability and disaster recovery

These standards apply to:

- PostgreSQL database servers
- Replication environments
- Backup systems
- Monitoring platforms
- DBA operational activities

All PostgreSQL environments must be periodically validated against these standards.

---

2. PostgreSQL Configuration Standards

2.1 Memory Configuration

Memory parameters must be configured according to server RAM and workload characteristics.

Improper memory settings can result in:

- Excessive disk I/O
- Poor query performance
- High latency
- Frequent cache misses
- System instability

Recommended Memory Configuration

Parameter	Recommended Value	Severity	Description
shared_buffers	25% of server RAM	HIGH	Controls PostgreSQL shared memory cache
work_mem	16MB – 64MB	MEDIUM	Used for sort/hash operations
maintenance_work_mem	256MB – 1GB	MEDIUM	Used for vacuum and index maintenance
effective_cache_size	50% – 75% of RAM	MEDIUM	Helps planner estimate OS cache
wal_buffers	16MB minimum	LOW	WAL buffering optimization

Example

For a server with 32GB RAM:

Parameter	Recommended
shared_buffers	8GB
effective_cache_size	24GB
work_mem	32MB

2.2 Connection Management

The number of PostgreSQL connections must be controlled carefully to avoid memory pressure and context switching overhead.

Best Practices

- Use connection pooling wherever possible
- Avoid very high max_connections values
- Monitor idle sessions regularly
- Configure application-level connection limits

Recommended Configuration

Parameter	Recommended Value	Severity
max_connections	Less than 500	HIGH
superuser_reserved_connections	5	MEDIUM
idle_in_transaction_session_timeout	5 minutes	HIGH

Validation Query

```
SHOW max_connections;
```

---

3. WAL & Checkpoint Management

Write Ahead Logging (WAL) is critical for durability, crash recovery, replication, and Point-In-Time Recovery (PITR).

Improper WAL configuration may result in:

- Slow recovery
- Excessive checkpoint spikes
- Replication lag
- Increased disk usage

Recommended WAL Settings

Parameter	Recommended Value	Severity
wal_level	replica	HIGH
archive_mode	on	HIGH
archive_command	Configured and tested	HIGH
max_wal_size	4GB or higher	MEDIUM
min_wal_size	1GB	LOW
checkpoint_timeout	15 minutes	MEDIUM

Example Validation

```
SHOW archive_mode;
SHOW wal_level;
SHOW checkpoint_timeout;
```

---

4. Autovacuum & Maintenance Standards

Autovacuum is mandatory in all PostgreSQL environments.

Disabling autovacuum can lead to:

- Table bloat
- Transaction ID wraparound
- Slow queries
- Increased storage usage

Mandatory Settings

Parameter	Expected Value	Severity
autovacuum	on	CRITICAL
autovacuum_max_workers	Minimum 3	MEDIUM
autovacuum_naptime	1 minute	LOW
vacuum_cost_limit	Tuned appropriately	LOW

Monitoring Query

```
SELECT
    relname,
    n_dead_tup,
    last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

---

5. Logging & Monitoring Standards

Logging must be enabled to support:

- Troubleshooting
- Performance analysis

- Security investigations
- Audit requirements

Recommended Logging Configuration

Parameter	Recommended Value	Severity
logging_collector	on	HIGH
log_min_duration_statement	1000 ms	HIGH
log_checkpoints	on	MEDIUM
log_connections	on	LOW
log_disconnections	on	LOW
log_line_prefix	Standardized format	MEDIUM

Standard Log Prefix

```
%m [%p] %u@%d %h
```

Example Query

```
SHOW logging_collector;
SHOW log_min_duration_statement;
```

---

6. Security Best Practices

All PostgreSQL environments must follow enterprise security standards.

Security Controls

Control	Requirement	Severity
password_encryption	scram-sha-256	HIGH
SSL	Enabled	HIGH
Public access	Restricted	CRITICAL
Superuser usage	Limited	HIGH
Default accounts	Reviewed regularly	MEDIUM

Access Management Guidelines

- Avoid direct application access using superuser accounts

- Rotate passwords periodically
- Enforce least privilege access
- Restrict pg_hba.conf entries
- Enable SSL encryption

Example Validation Query

```
SHOW password_encryption;
SHOW ssl;
```

---

7. Backup & Recovery Standards

Backups are mandatory for all environments.

Backup validation must include periodic recovery testing.

Backup Standards

Category	Requirement	Frequency
Full Backup	Mandatory	Daily
WAL Archiving	Mandatory	Continuous
PITR Testing	Mandatory	Monthly
Backup Retention	Minimum 30 Days	Standard

Validation Checklist

Validation Item	Status
Full backup successful	Required
WAL archiving active	Required
Restore tested	Required
Recovery documented	Required

---

8. Replication & High Availability

Production databases must support high availability architecture.

Recommended Practices

- Use streaming replication
- Monitor replication lag
- Configure replication slots properly
- Test failover procedures regularly

Replication Validation

Parameter	Expected Value
wal_level	replica
hot_standby	on
max_wal_senders	Minimum 5
max_replication_slots	Configured

Monitoring Query

```
SELECT
    application_name,
    state,
    sync_state,
    write_lag,
    replay_lag
FROM pg_stat_replication;
```

---

9. Indexing Best Practices

Indexes must be reviewed regularly to ensure optimal query performance.

Best Practices

- Remove unused indexes
- Avoid excessive indexing
- Monitor index bloat
- Analyze slow queries periodically

Validation Queries

Unused Indexes

```
SELECT
    schemaname,
    relname,
```

```
    indexrelname,
    idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0;
```

Large Tables

```
SELECT
    relname,
    pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

---

10. Compliance Evaluation Matrix

Category	Validation Method	Compliance Type
Configuration	SHOW commands	Automated
Security	Parameter validation	Automated
Replication	pg_stat_replication	Automated
Backup	Operational verification	Manual + Automated
Maintenance	pg_stat_user_tables	Automated
Monitoring	Log validation	Automated

---

11. Severity Classification

Severity	Meaning	Action Required
CRITICAL	Major operational/security risk	Immediate
HIGH	Significant performance/reliability risk	Within 24 hours
MEDIUM	Operational improvement recommended	Planned remediation
LOW	Optimization recommendation	Best effort

---

12. Sample Compliance Report

Example Findings

Rule	Expected	Actual	Status	Severity
shared_buffers	25% RAM	128MB	FAIL	HIGH
autovacuum	on	off	FAIL	CRITICAL
archive_mode	on	on	PASS	LOW
log_min_duration_statement	1000	-1	FAIL	MEDIUM
max_connections	< 500	1000	FAIL	HIGH

---

13. AI-Assisted Recommendation Example

Finding

`autovacuum = off`

AI Recommendation

Autovacuum is currently disabled. This can lead to severe table bloat, transaction ID wraparound risks, and degraded query performance. It is recommended to enable autovacuum immediately and monitor dead tuple accumulation using pg_stat_user_tables.

---

14. Operational Checklist

Daily DBA Checklist

Activity	Status
Check replication lag	Required
Review backup success	Required
Monitor disk usage	Required
Review blocking sessions	Required
Analyze slow queries	Recommended

---

15. Conclusion

This document defines the baseline PostgreSQL operational and configuration standards for enterprise environments.

Periodic compliance validation must be performed to:

- maintain database health
- improve reliability
- ensure security compliance
- reduce operational incidents
- standardize PostgreSQL deployments

The AI-assisted PostgreSQL Compliance Validator platform can automate these checks and generate intelligent recommendations for DBA teams.